TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

TECHNICAL REPORT

Secure Code Debugging and Review Assistant

Field	Details
Institute / University	PCCOE (confirm official institute name)
Programme / Batch	Applied AI/ML Capstone — TechPulse FY-26
Student Name	_____
PRN	_____
Case Study ID (CS1–CS5)	Case Study 4 (confirm official ID)
Project Title	Secure Code Debugging and Review Assistant

1. Problem Statement and Intended Users

Manual code review can be time-consuming, and developers may overlook security weaknesses or code-quality issues. Static-analysis tools identify suspicious patterns but do not always provide enough contextual guidance for a developer to understand and remediate a finding. This project combines Python static analysis with retrieval of secure-coding guidelines and a downloadable report to support preliminary code review.

Intended users: students, Python developers, reviewers, and educators who need an accessible first-pass review tool. The application is advisory and does not replace manual review, testing, or a professional security audit.

2. Input Data / Knowledge Base

Item	Description
Sources	Local Markdown/TXT files under data/guidelines/. Current demo source: coding_guidelines_demo.md.
Size and format	Small, locally maintained secure-coding guideline document in Markdown; indexed into 11 chunks in the observed run.
Preprocessing / EDA done	Read text as UTF-8 with replacement for decoding errors; split Markdown content into sections; normalize each section to whitespace-separated text before chunking.
Chunking strategy	Section-aware chunking, up to 180 words per chunk, without cross-section overlap in the updated implementation. Chunk metadata stores source path and chunk number.

3. Solution Architecture

Component	Choice (name, version, hosting)
Pre-trained model	No general-purpose model required for static analysis. Optional OpenAI model configured through OPENAI_MODEL; default in code is gpt-4.1-mini, not tested in current run.
Embedding model	sentence-transformers/all-MiniLM-L6-v2 via Sentence Transformers embedding function.
Vector store	ChromaDB PersistentClient; local persistence directory chroma_db; collection coding_guidelines.
RAG pipeline / AI-agent design	Retrieval-Augmented Generation-style pipeline: finding-aware search queries retrieve relevant local guideline passages. The current review workflow is not a multi-agent system.
Service layer and UI	Python application with Streamlit UI in app.py; report generation in report_generator.py.
Tools / deterministic rules	Bandit and Ruff for static analysis; deterministic guideline indexing/retrieval; Markdown report export.

System architecture diagram

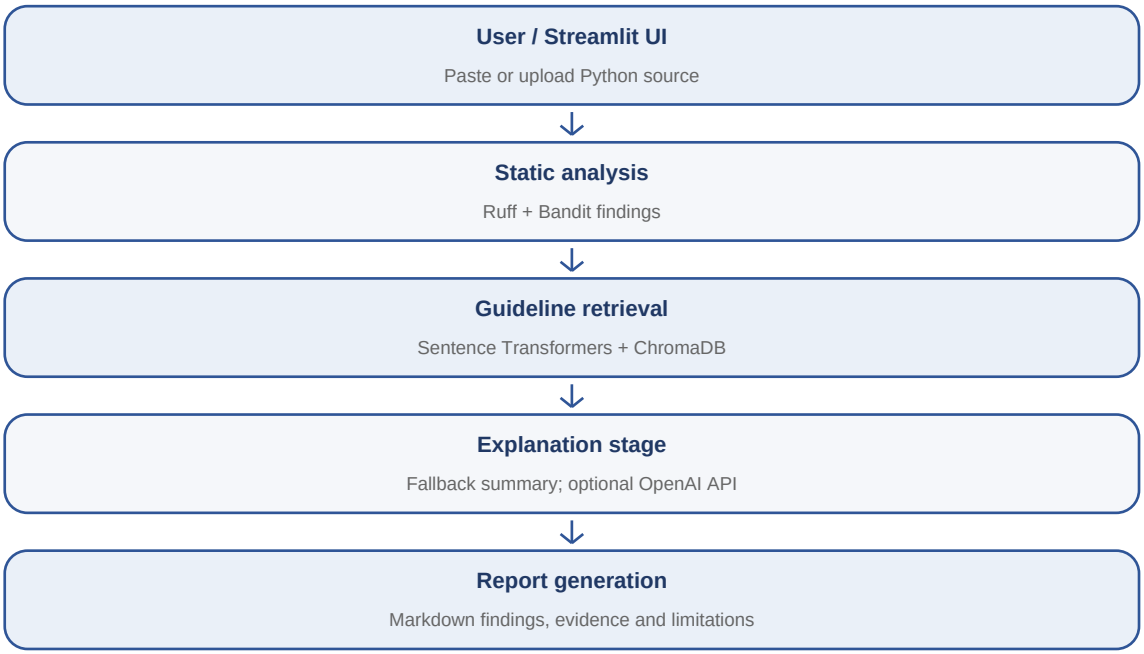

Figure 1. High-level architecture of the Secure Code Review Assistant.

4. Implementation and Configuration

Application entry point: app.py accepts Python source, runs analysis, retrieves guidelines, displays results, and exposes report download functionality.

Static analysis: review_engine.py invokes Ruff and Bandit when available and collects findings with tool name, rule ID, severity, line number, evidence, and recommendation. Syntax errors are also surfaced by the parser path.

Knowledge base: rag_pipeline.py reads local .md/.txt guideline files, splits them into section-aware chunks, embeds and stores them in ChromaDB, and queries the stored collection for relevant passages. The index must be refreshed after guideline content or chunking logic changes.

Explanation and report: when OPENAI_API_KEY is absent, the system returns a fallback summary and explicitly notes that the LLM is not configured. When configured, the code can call the OpenAI Responses API. report_generator.py produces a Markdown report.

Configuration and privacy: the optional API key and model are loaded from a local .env file. Do not commit .env, real credentials, or proprietary source code. The current app was tested locally using Python 3.11 and Streamlit.

Implementation workflow

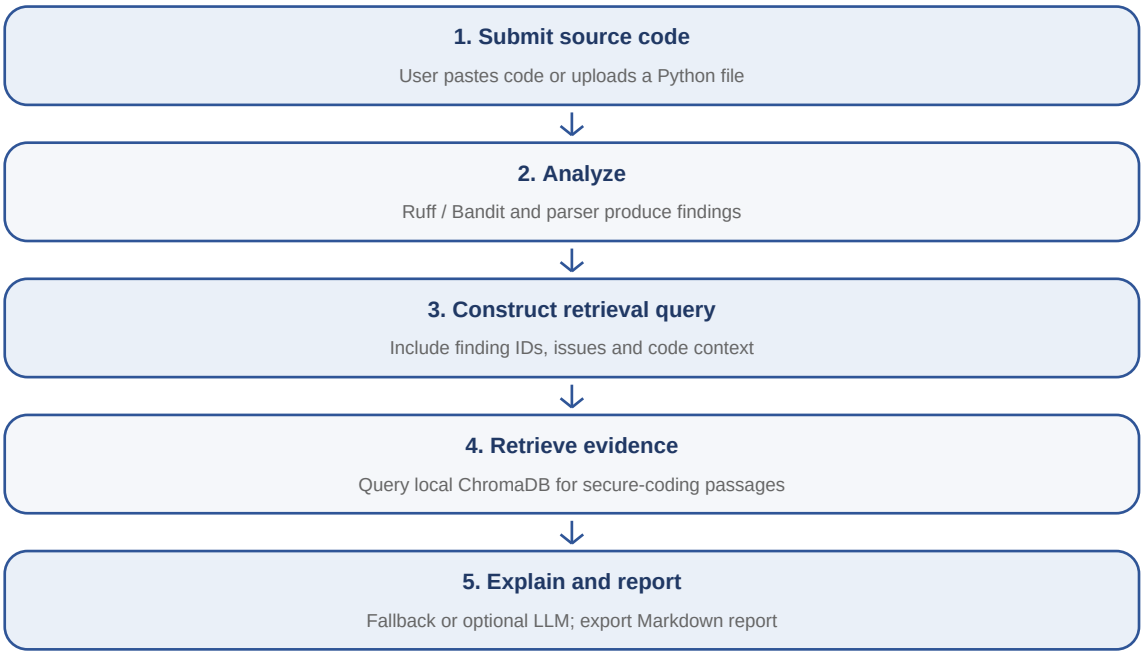

Figure 2. Workflow followed for each code review.

5. Evaluation Steps and Results

The evaluation set contains six manually recorded scenarios. The results below reflect the observed local runs and the supplied evaluation CSV. TC01 and TC02 reuse the same vulnerable sample, so they are not independent test inputs.

#	Test question / scenario	Expected answer / behaviour	Result (Pass/Fail, notes)
1	TC01: Hardcoded demo credential and risky subprocess call	Detect B105, B404 and B602 in sample_vulnerable.py	Pass — 3 findings recorded
2	TC02: Subprocess shell usage	Detect Bandit B602 for shell=True	Pass — B602 reported; overlaps TC01
3	TC03: Clean calculate_total function	No findings for the simple validated function	Pass — 0 findings
4	TC04: Invalid Python syntax	Report missing colon / syntax error at line 1	Pass — 1 syntax finding
5	TC05: Safer subprocess with shell=False	B602 absent; other analyzer warnings may remain	Pass — B404, B607 and B603 reported
6	TC06: Allowlisted user role	No static-analysis findings for allowlisted role sample	Pass — 0 findings

Metrics and observations

Metric	Score / observation
Test-case pass rate	6/6 recorded test cases marked Pass in evaluation/test_cases.csv. This is not a measured detection accuracy.
Retrieval relevance	For the vulnerable sample, the observed evidence included credential guidance (relevance 0.67), subprocess guidance (0.67), and cryptography guidance (0.55). The third passage was less directly relevant.
Latency	Not formally measured in the current evaluation.
Precision / recall	Not calculated; a larger labelled dataset and ground-truth labels are required.

6. Responsible AI Measures

Area	Measure adopted in this project
Grounding and citations	The report includes source file and chunk identifiers for retrieved guideline passages. Retrieval evidence is advisory and should be checked against the actual code.
Privacy	Guideline storage and retrieval are local. The optional OpenAI API may transmit submitted code to an external provider; do not submit confidential or proprietary code without authorization.
Security	Keep API keys in .env, exclude secrets from version control, and rotate any credential that may have been exposed. Do not treat analyzer output as proof of exploitability.
Human oversight / review	Developers must inspect the evidence, validate fixes, and run tests. AI-generated content is not claimed to be tested unless test evidence exists.

7. Innovation Highlights

• Finding-aware retrieval queries: the retrieval query can include analyzer rule IDs and recommendations, rather than relying only on raw source code.

• Section-aware guideline chunks: headings such as Secure Subprocess Execution and Secrets and Credential Management are retained with their passages to improve retrieval relevance.

• Evidence-oriented reports: findings are exported with severity, line number, rule ID, code evidence, guideline passages, and explicit limitations.

These are implementation features demonstrated in the current prototype; no autonomous agent orchestration or automatic code patching is claimed.

8. Screenshots, Logs and Evidence

Insert the actual screenshots from your local application in the final submission. The captions below identify useful evidence to include.

Figure 3 — Vulnerable-code review: Streamlit screen showing three findings: Bandit B404 (Low), B105 (Low), and B602 (High).

Figure 4 — Retrieved guideline evidence: Output showing Secrets and Credential Management and Secure Subprocess Execution passages with relevance scores of 0.67, plus a less directly relevant Cryptography passage at 0.55.

Figure 5 — Clean-code test: Review of calculate_total() showing zero findings.

Figure 6 — Exported report: Markdown report for vulnerable code containing the three findings, relevant guideline evidence, fallback LLM notice, and limitations.

9. Reflection and Conclusions

The prototype demonstrates a practical combination of deterministic static analysis and retrieval of secure-coding guidance. Testing showed that Bandit identified a hardcoded demo credential and the use of shell=True, while a simple clean function returned no findings. Section-aware chunking and finding-aware retrieval improved the relevance of evidence for the vulnerable sample. The downloadable Markdown report provides a repeatable artifact for review and evaluation.

Challenges included configuring a compatible Python environment, maintaining the local ChromaDB index after changing chunking logic, and tuning retrieval so that guidance matches the detected issue. The optional LLM remains unconfigured in the evaluated run, retrieval metrics and latency have not been formally measured, and the six-case evaluation set is small. Future work should configure and evaluate the LLM, expand labelled test data, calculate precision and recall, improve retrieval ranking, and add automated regression tests.

10. AI-Tool Usage Declaration

AI tool / model	Purpose	Artifact affected	Verified by Student (Y/N)
ChatGPT / OpenAI assistant	Drafting, debugging guidance, report wording, and retrieval troubleshooting	Code changes and this technical report; student must review and verify	_____
Sentence Transformers: all-MiniLM-L6-v2	Embedding guideline chunks and retrieval queries	Local RAG retrieval pipeline	Y (local retrieval tested)
OpenAI API model (optional)	LLM-generated review explanation when configured	Optional explanation function	N — not configured in observed test

11. Student Declaration

I confirm that this report represents my own understanding and contribution, that the work follows the approved case-study scope, and that I can explain every significant part of the implementation. I will review and correct all generated content, verify technical claims against my code and test evidence, and complete the AI-tool usage declaration accurately before submission.

Signature: _____	Date: _____
Name: _____	PRN: _____

Before submission: confirm the official institute name, student name, PRN, and case-study ID; insert real screenshots; verify the template requirements and references; and ensure all claims match your final code and evaluation.